Univerza v Mariboru
Fakulteta za naravoslovje in matematiko | Oddelek za fiziko

LABORATORIJSKI DELOVNI LIST 1

Numerično reševanje nelinearnih algebrajskih enačb

Programsko okolje: Python

Vse računalniške naloge te vaje se izvajajo **izključno v programskem jeziku Python** (priporočena različica 3.10 ali novejša). Uporabljamo knjižnici `numpy` (numerično računanje) in `matplotlib` (grafični prikaz). Uporaba drugih programskih jezikov ali okolij (npr. MATLAB, Octave, Mathematica) ni predvidena. Rešitev oddajte kot Python skripto (`.py`) ali Jupyter zvezek (`.ipynb`).

1 Teoretične osnove in motivacija

Pri preučevanju fizikalnih dinamičnih sistemov pogosto naletimo na določanje ravnotežnih stanj ali karakterističnih parametrov, kjer analitična rešitev v zaprti obliki ne obstaja. Vsako takšno nelinearno enačbo preuredimo v standardno ničelno obliko:

$$f(x) = 0. \tag{1}$$

Naloga numeričnih postopkov je določitev ničle x^*, pri kateri je $f(x^*) = 0$. V okviru te vaje primerjamo tri standardne iterativne metode:

1. **Metoda bisekcije (intervalna metoda):** Temelji na izreku o zveznosti funkcije. Če je funkcija zvezna na intervalu $[a, b]$ in velja pogoj menjave predznaka:

   $$f(a) \cdot f(b) < 0, \tag{2}$$

   potem vmes obstaja vsaj ena realna ničla. Interval v vsakem koraku razpolovimo s točko $c = \frac{a+b}{2}$ ter zožimo iskalno območje glede na predznak produkta $f(a) \cdot f(c)$. Metoda je robustna in brezpogojno konvergentna, vendar linearno počasna.

2. **Newton–Raphsonova metoda (tangentna metoda):** Izhaja iz linearizacije funkcije s prvim členom Taylorjevega razvoja v okolici trenutnega približka x_k:

   $$f(x) \approx f(x_k) + f'(x_k)(x - x_k) = 0 \implies x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}. \tag{3}$$

   Metoda v bližini enostavne ničle izkazuje kvadratno hitrost konvergence, vendar zahteva poznavanje prvega odvoda $f'(x)$ ter ustrezen začetni približek x_0, pri čemer $f'(x_k) \neq 0$.

3. **Sekantna metoda:** Odvod $f'(x_k)$ nadomesti s končno diferenco skozi zadnja dva iteracijska približka:

   $$x_{k+1} = x_k - f(x_k)\frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}. \tag{4}$$

   Metoda ne zahteva računanja analitičnega odvoda, njena stopnja konvergence pa znaša približno $\approx 1{,}618$.

2 Pripravljalna naloga (analitični del)

Obravnavamo transcendentno enačbo, ki opisuje stacionarno lego v nelinearnem potencialu:

$$x - 2\sin(x) = 0{,}5 \iff f(x) = x - 2\sin(x) - 0{,}5 = 0. \tag{5}$$

Naloga:

1. Izračunajte prvi odvod funkcije: $f'(x) =$ _____

2. Preverite vrednosti na robovih intervala $[a, b] = [2{,}0,\ 2{,}5]$:

   - $f(2{,}0) = 2{,}0 - 2\sin(2{,}0) - 0{,}5 \approx$ _____ (predznak: _____)
   - $f(2{,}5) = 2{,}5 - 2\sin(2{,}5) - 0{,}5 \approx$ _____ (predznak: _____)

   Ali je na danem intervalu izpolnjen pogoj za zagon metode bisekcije? **DA / NE**

3 Laboratorijske naloge na računalniku (Python)

Vse naloge v tem razdelku rešite v programskem jeziku **Python**.

3.1 Naloga 1: Samostojna implementacija metod (40 min)

V programskem jeziku **Python** brez uporabe specializiranih knjižnic za optimizacijo (npr. brez `scipy.optimize`) implementirajte naslednje funkcije:

- `bisekcija(f, a, b, tol, max_iter)`
- `newton_raphson(f, df, x0, tol, max_iter)`
- `sekantna(f, x0, x1, tol, max_iter)`

Vsaka funkcija naj spremlja absolutno vrednost odstopanja $|f(x_k)|$ skozi iteracije ter vrne izračunano ničlo x^*, število korakov in seznam napak.

3.2 Naloga 2: Primerjava natančnosti in hitrosti konvergence (25 min)

Poiščite ničlo funkcije $f(x)$ za dve zahtevani toleranci: $\text{tol} = 10^{-6}$ in $\text{tol} = 10^{-12}$. Uporabite začetne pogoje: $a = 2{,}0$, $b = 2{,}5$; $x_0 = 2{,}0$; $x_1 = 2{,}5$. Rezultate vpišite v spodnjo preglednico.

Metoda	**Ničla x^* (tol $= 10^{-6}$)**	**Št. korakov**	**Ničla x^* (tol $= 10^{-12}$)**	**Št. korakov**
Bisekcija				
Newton–Raphson				
Sekantna metoda				

Tabela 1: Primerjalna analiza numeričnih metod za iskanje ničel.

3.3 Naloga 3: Grafični prikaz in analiza napake (25 min)

1. Narišite graf $f(x)$ na intervalu $x \in [1{,}5, 3{,}0]$, dodajte premico $y = 0$ in označite najdeno ničlo.

2. Na skupnem grafu z **logaritemsko skalo na osi y** (`plt.yscale('log')`) prikažite potek napake $|f(x_k)|$ v odvisnosti od iteracije k za vse tri metode. Primerjajte strmino padanja krivulj.

4 Začetna programska predloga (Python)

```python
import numpy as np
import matplotlib.pyplot as plt

def f(x):
    return x - 2.0 * np.sin(x) - 0.5

def df(x):
    return 1.0 - 2.0 * np.cos(x)

def bisekcija(f, a, b, tol=1e-12, max_iter=100):
    if f(a) * f(b) > 0:
        raise ValueError("Funkcija nima nasprotnih predznakov na robovih!")
    errs = []
    for k in range(max_iter):
        c = 0.5 * (a + b)
        fc = f(c)
        errs.append(abs(fc))
        if abs(fc) < tol or 0.5 * (b - a) < tol:
            return c, k + 1, errs
        if f(a) * fc < 0:
            b = c
        else:
            a = c
    return c, max_iter, errs

def newton_raphson(f, df, x0, tol=1e-12, max_iter=100):
    x = x0
    errs = []
    for k in range(max_iter):
        fx = f(x)
        errs.append(abs(fx))
        if abs(fx) < tol:
            return x, k + 1, errs
        dfx = df(x)
        if abs(dfx) < 1e-14:
            raise ZeroDivisionError("Odvod f'(x) je blizu nicli!")
        x = x - fx / dfx
    return x, max_iter, errs

# Izvedba izracuna za toleranco 1e-12
tol = 1e-12
x_bis, it_bis, err_bis = bisekcija(f, 2.0, 2.5, tol=tol)
x_new, it_new, err_new = newton_raphson(f, df, 2.0, tol=tol)

print(f"Bisekcija:      x* = {x_bis:.12f}, iteracij = {it_bis}")
print(f"Newton-Raphson: x* = {x_new:.12f}, iteracij = {it_new}")
```

5 Vprašanja za razpravo in zaključni zagovor

Razmislek 1: Vpliv lokalnih ekstremov

Kaj se zgodi z Newton–Raphsonovo iteracijo, če za začetni približek izberemo točko, kjer velja $f'(x_0) \approx 0$? Kakšno je geometrijsko obnašanje tangente v tem primeru?

Razmislek 2: Analiza računske zahtevnosti

Zakaj je število potrebnih iteracij pri metodi bisekcije povsem neodvisno od nelinearne oblike funkcije $f(x)$ in odvisno izključno od začetne širine intervala $|b - a|$ ter tolerance ε? Izpeljite formulo za minimalno število korakov N.

Razmislek 3: Praktična uporaba v fiziki

V katerih fizikalnih problemih (npr. pri reševanju implicitnih diferencialnih enačb ali pri funkcijah, ki so rezultat »black-box« numerične simulacije) je sekantna metoda primernejša od Newton–Raphsonove metode?